

第 5 章

带上眼罩测试软件

好，精彩的来了。本章讲述大多数人想像的软件测试。现在放下二郎腿，在计算机前面坐好，开始寻找缺陷。

这对于软件测试新手可能是分配的第一项任务。如果应聘软件测试职位，主考人一定会问如何测试新软件程序或者程序新功能。

马上进入状态，猛敲键盘去找破绽是非常轻松的事。这种方法可能会顶用一小会儿。如果软件处于开发阶段，很容易幸运地迅速找出一些软件缺陷。遗憾的是，这样轻松的成功很快就无法再现了，要成为一个成功的软件测试员，需要采用更结构化的、目标明确的方法继续测试。

本章描述最常用、最有效的软件测试技术。无论测试何种类型的程序——公司的财务软件包、工业自动化程序还是市面上流行的射击游戏，这些技术都适用。

使用这些技术不必成为程序员。尽管这些技术都基于编程基本原理，但是不要求编写代码。有一些技术需要一点背景知识来解释该技术为什么有效，但是本章所有代码实例都很短小，且用宏语言来编写，使演示容易理解。假如读者已经涉足编程，想学习更多低层测试技术，那么在读完本章之后可以转到第6章“检查代码”和第7章“带上X光眼镜测试软件”学习白盒测试。

本章重点包括：

- 动态黑盒测试是什么
- 如何通过等价类划分减少测试用例的数量
- 如何判别故障边界条件
- 使用良好数据引入缺陷
- 如何测试软件状态和状态转换
- 如何使用重复、压迫和重负的方法找出缺陷
- 缺陷的一些秘密隐藏之处

5.1 动态黑盒测试：带上眼罩测试软件

不深入代码细节测试软件的方法称为动态黑盒测试。它是动态（dynamic）的，因为程序在运行——软件测试员像用户一样使用它；同时，它是黑盒子（black-box），因为测试时不知道程序如何工作——带上了眼罩。测试员输入数据、接受输出、检验结果。动态黑盒测试常常被称为行为测试，因为测试的是软件在使用过程中的实际行为。

有效的动态测试需要关于软件行为的一些定义——也即需求文档或者产品说明书。不必了解软件“盒子”内发生的事情——而只需知道输入A输出B或执行操作C得到结果D。好的产品说明书会提供这些细节信息。

清楚了被测试软件的输入和输出之后，接下来要开始定义测试用例（test case）。测试用例是指进行测试时使用的特定输入，以及测试软件的过程步骤。图5-1给出了用于Windows计算器加法功能的一些用例。

Windows计算器加法测试用例		
0+0	应等于	0
0+1	应等于	1
254+1	应等于	255
255+1	应等于	256
256+1	应等于	257
1022+1	应等于	1023
1023+1	应等于	1024
1024+1	应等于	1025
...		
...		

图5-1 测试用例给出了各种输入以及测试程序的步骤

注意 选择测试用例是软件测试员最重要的一项任务。不正确的选择可能导致测试量过大或者过小，甚至测试目标不对。准确评估风险，把无穷尽的可能性减少到可以控制的范围是成功的诀窍。

本章后面和本书余下的大部分将讲述选择合适测试用例的策略。第18章“编写和跟踪测试用例”探讨编写和管理测试用例的技术。

在没有产品说明书时使用探索测试

专业的、成熟的软件开发过程都会有软件的详细说明。如果采用大爆炸模式或者边写边改模式，作为测试依据的产品说明书可能没有。尽管这对于软件测试员不是理想的状况，但是此时可以采取称为探索测试的解决方案——了解软件、设计测试、执行测试同时进行。

这就需要把软件当做产品说明书来对待。系统地逐项了解软件的功能、记录软件的执行情况、详细描述功能，运用第4章“检查产品说明书”中所讲的静态黑盒技术，把软件当成说明书来分析，然后运用本章所讲的动态黑盒技术进行测试。

在这种情况下，无法像有产品说明书那样完整测试软件——比如无法断定是否遗漏功能，但是可以系统地测试软件，找到软件缺陷几乎是肯定的。

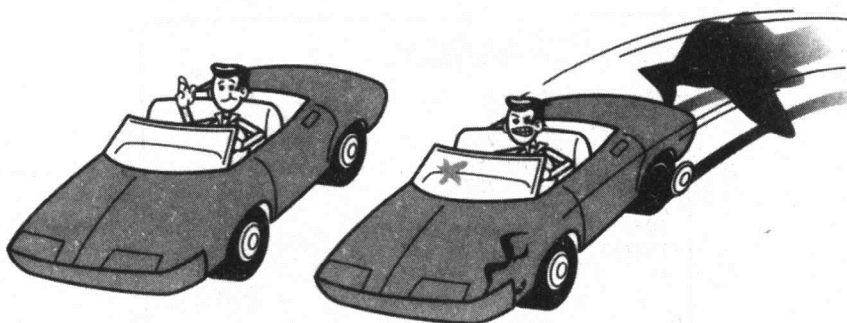
5.2 通过性测试和失效性测试

测试软件有两种基本方法：通过性测试（test-to-pass）和失效性测试（test-to-fail）。在进行通过性测试时，实际上是确认软件至少能做什么，而不会考验其能力。软件测试员并不需要想尽办法让软件崩溃，仅仅运用最简单、最直观的测试用例。

既然软件测试的目标是找出软件缺陷，为什么还要进行通过性测试呢？为什么不尽量去

设法找出软件缺陷呢？不，开始不是这样的。

设想一种类似的情况，即一辆新设计的汽车（见图5-2）、如果受命测试刚下生产线、从没开过的一辆样车，测试者可能不会立即坐上去、发动汽车、驶入检测道路、尽全力高速行驶。这样可能会撞车，发生生命危险。作为新车，在正常驾驶条件下低速行驶时可能会暴露所有的缺陷。也许轮胎尺寸不对，或者制动力不足，或者发动机噪声过大。在上路冲击极限速度之前是可以发现并解决这些问题的。



通过性测试

失效性测试

图5-2 在失效性测试之前利用通过性测试找出缺陷

注意 在设计和执行测试用例时，总是首先进行通过性测试。在破坏性测试之前看看软件基本功能是否能实现是很重要的，软件测试员可能会吃惊地发现仅仅正常使用软件就会发现那么多软件缺陷。

确信软件在普通情况下能正确运行之后，就可以采取各种手段搞垮软件来找出软件缺陷了。纯粹为了破坏软件而设计和执行的测试用例称为失效性测试或错误强制测试。本章后面将会讲到失效性测试通常不会突然出现。虽然看起来与通过性测试差不多，但是它是蓄意攻击软件的薄弱环节。

错误提示信息：是通过性测试还是失效性测试

测试用例中常见的一种就是设法迫使软件出现错误提示信息。大家熟知其中的一些——例如没有在软驱中插入磁盘而向软盘中保存文件。这些用例实际上搅乱了通过性测试和失效性测试之间的界限。产品说明书可能会特别说明某些输入条件将产生错误提示信息。这似乎显然是通过性测试用例，但是由于迫使软件出错，因此也可视为失效性测试。实际上，可能两者都是。

不必费力去区分它们。重要的是设法迫使指定的错误信息出现，或者设计测试用例迫使未考虑到的错误暴露出来。最终可能在通过性测试和失效性测试中都找出软件缺陷。

5.3 等价类划分

选择测试用例是软件测试员最重要的任务。选择测试用例的方法是等价类划分（equivalence partitioning），有时称为等价分类（equivalence classing）。等价类划分是指分步骤地把海量（无限）的测试用例集减得很小，但过程同样有效。

如果要测试Copy命令，可以把这5种输入划分为3个：单击菜单命令，键入c或Ctrl+c。对软件质量更有信心之后，就会知道无论以何种方式激活Copy功能都工作正常，甚至可以把这些划分进一步缩减为1个，例如按Ctrl+c组合键。

- 作为第3个例子，看一下在标准的Save As对话框（见图5-4）中输入文件名称的情形。

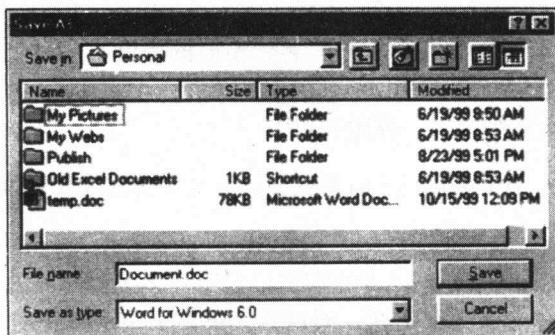


图5-4 Save As对话框中的File Name 文本框显示了几种等价划分的可能性

Windows文件名可以包含除了、/、*、?、“<”、“>和\之外的任意字符。文件名长度是1~255个字符。如果为文件名创建测试用例，等价划分有合法字符、非法字符、合法长度的名称、长度过长名称和长度过短名称。

请记住，等价类划分的目标是把可能的测试用例集缩减到可控制且仍然足以测试软件的小范围内。因为选择了不完全测试，就要冒一定的风险，所以选择分类时必须仔细。

注意 如果为了减少测试用例的数量过度划分等价类，就有漏掉那些可能暴露软件缺陷的测试的风险。对于初涉软件测试者，一定要请经验丰富的测试员审查划分好的等价类别。

关于等价类划分最后要讲的一点是等价类划分可能主观。科学有时也是一门艺术。测试同一个复杂程序的两个软件测试员可能会得出两组不同的等价划分间。只要审查等价划分的人认为它们足以覆盖测试对象就行了。

5.4 数据测试

对软件最简单的认识就是将其分成两部分：数据（或其范围）和程序。数据包括键盘输入、鼠标单击、磁盘文件、打印输出等。程序是指可执行的流程、转换、逻辑和运算。软件测试常用的一个方法是把测试工作按同样的形式划分。

对数据进行软件测试，就是在检查用户输入的信息、返回的结果以及中间计算结果是否正确。

数据的例子如下：

- 在文字处理程序中输入的文字。
- 电子表格中输入的数字。
- 太空游戏中余下的射击次数。
- 图像处理软件打印的图片。

- 存放在软盘中的备份文件。
- 通过调制解调器在电话线上发送的数据。

即使最简单的程序要处理的数据量也可能极大。还记得在计算器上执行简单加法的输入数据的全部可能性吗？再想一想文字处理程序、导弹制导系统软件和证券交易程序。使所有这些数据得以测试的技巧（如果称得上的话）是，根据一些关键的原则进行等价类划分，以合理减少测试用例，这些关键的原则是：边界条件、次边界条件、空值和无效数据。

5.4.1 边界条件

描述边界条件（boundary condition）测试的最佳方式如图5-5所示。如果在悬崖峭壁边可以自信而安全地行走而不掉下去，平地就几乎不在话下了。如果软件能在其边界运行，那么在正常情况下就应该不会有什么问题。

边界条件是特殊情况，因为编程从根本上说在边界上容易产生问题。软件是很极端的——即要么对要么不对。令人奇怪的是如果对一定范围的数据进行操作，程序员往往在处理大量中间数值时都是对的，但是可能在边界处出现错误。清单5-1说明了在一个极简单的程序中是如何产生边界条件问题的。

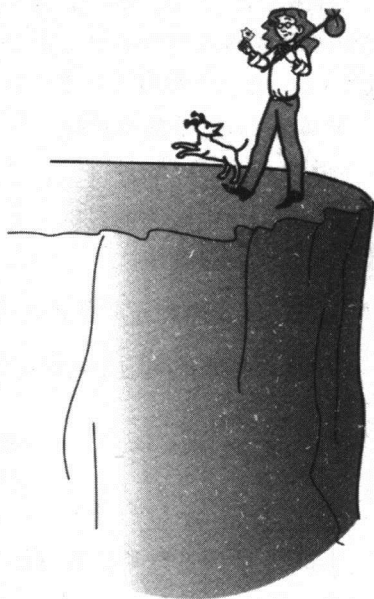


图5-5 软件边界与悬崖很类似

清单5-1 演示边界条件缺陷的简单BASIC程序

```

1: Rem Create a 10 element integer array
2: Rem Initialize each element to -1
3: Dim data(10) As Integer
4: Dim i As Integer
5: For i = 1 To 10
6:   data(i) = -1
7: Next i
8: End

```

这段代码的目的是创建包含10个元素的数组，并为数组中的每一个元素赋初值-1。看起来相当简单。它建立了包含10个整数的数组data和一个计数值i。For循环从1到10，数组中从第1个元素到第10个元素被赋予数值-1。边界问题在哪儿？

在大多数BASIC脚本中，当以声明的范围定义数组大小时——在本例中定义语句是Dim data (10) as Integer——第一个创建的元素是0，而不是1。该程序实际上创建了一个从data (0)到 data(10)共11个元素的数组。程序从1到10循环将数组元素的值初始化为-1，但是由于数组的第一个元素是data(0)，因此它没有被初始化。程序执行完毕，数组值如下：

```

data(0) = 0      data(6) = -1
data(1) = -1     data(7) = -1
data(2) = -1     data(8) = -1

```

```
data(3) = -1      data(9) = -1
data(4) = -1      data(10) = -1
data(5) = -1
```

注意data (0) 的值是0, 而不是-1。如果这位程序员以后忘记了, 或者其他程序员不知道这个数组是如何初始化的, 那么他就可能会用到数组的第1个元素 data (0), 以为它的值设置成了-1。诸如此类的问题很常见, 在复杂的大型软件中, 可能导致极其讨厌的软件缺陷。

5.4.1.1 边界条件类型

现在我们要仔细探讨边界是由什么构成的。测试新手常常意识不到一组给定的数据包含多少边界。虽然较为明显的通常不多, 但是深入挖掘可以找到更多不明显的、有趣的和常常导致软件缺陷的边界。

注意 边界条件是指软件运行在计划操作界限的边界的情况。

如果软件测试问题包含确定的边界, 那么看看以下的数据类型:

数值	速度
字符	地点
位置	尺寸
数量	

同时, 考虑这些类型的下述特征:

第一个 / 最后一个	最小值 / 最大值
开始 / 完成	超过 / 在内
空 / 满	最短 / 最长
最慢 / 最快	最早 / 最迟
最大 / 最小	最高 / 最低
相邻 / 最远	

这些绝不是确定的列表, 而是一些可能出现的边界条件。每一个软件测试问题各不相同, 可能包含各种不同的数据以及其独特的边界。

技巧 如果要选择在等价划分中包含哪些数据, 就根据边界来选择。

5.4.1.2 测试边界

到目前为止, 我们知道了要为软件操作的各种数据集合建立等价划分。由于软件容易在边界上产生缺陷, 因此, 如果要从等价划分中选择包含的数据, 从边界条件中选择会找出更多的软件缺陷。

然而, 仅仅测试边界线上的数据点往往不够充分。就像变戏法所说的那样 (“右手放进去, 右手伸出来, 右手放进去, 摇一摇……”), 最好测试一下边界的两边——往上再摇一点。

如果建立两个等价划分就可以找出更多软件缺陷。第一个划分包含认为应该正确的数据——在边界内部最后一两个合法的数据点。第二个区间包含认为可能出现错误的的数据——边界之外——一到两个非法的数据点。

技巧 提出边界条件时, 一定要测试临近边界的有效数据, 测试最后一个可能有效的数据, 同时测试刚超过边界的无效数据。

越界测试的做法通常是简单地对于最大值加1或者很小的数，以及对于最小值减1或者很小的数，例如：

- 第一个减1/最后一个加1。
- 开始减1/完成加1。
- 空了再减/满了再加。
- 慢上加慢/快上加快
- 最大数加1/最小数减1。
- 最小值减1/最大值加1。
- 刚好超过/刚好在内。
- 短了再短/长了再长。
- 早了更早/晚了更晚。
- 最高加1/最低减1。

看以下几个例子以便通盘考虑所有可能的边界：

- 如果文本输入域允许输入1~255个字符，就尝试输入1个字符和255个字符代表合法划分的数据。还可以输入254个字符作为合法输入。输入0个字符和256个字符代表非法划分的数据。
- 如果程序读写CD-R，就尝试保存一个尺寸极小，甚至只有一项的文件，然后保存一个很大的——刚好在光盘容量限制之内的文件。还要尝试保存一个空文件和一个尺寸大于光盘容量的文件。
- 如果程序允许在一张纸上打印多个页面，就尝试只打印一页（标准情况），并尝试打印所允许的最多页面。如果可能，还要尝试打印0页和比最多允许页面数多一页的页面。
- 也许软件有一个输入9位邮政编码的数据输入域。尝试输入00000-0000，即最小、最简单的值。尝试输入99999-9999，即最大的值。总之，尝试输入比允许范围大一点或者小一点的值。
- 如果测试飞行模拟程序，尝试控制飞机正好在地平线上以及最大允许高度上飞行。尝试在地平线和海平面之下飞行，以及在外太空飞行。

由于不可能对每一种情况完全测试，因此像上述例子一样，围绕边界条件进行等价划分，并建立测试用例是至关重要的。这是减小测试工作量最为有效的方法。

注意 在软件的每一个部分不断寻找边界是极为重要的，寻找做得越多，边界就会发现得越多，可能找出得软件缺陷就越多。

注意 缓冲区溢出（buffer overrun）是由边界条件缺陷引起的，它是造成软件安全问题的头号原因。第13章“软件安全性测试”讨论引起缓冲区溢出的特定条件以及如何进行测试。

5.4.2 次边界条件

上面讨论的普通边界条件是最容易找到的。它们在产品说明书中有定义，或者在使用软件的过程中明显。而有些边界在软件内部，最终用户几乎看不到，但是软件测试员仍有必要

进行检查。这样的边界条件称为次边界条件 (sub-boundary conditions) 或者内部边界条件 (internal boundary conditions)。

寻找这样的边界不要求软件测试员成为程序员或者具有阅读源代码的能力, 但是确实要求大体了解软件的工作方式。2的幂和ASCII表是这方面的两个例子。所测试的软件可能有许多其他的次边界条件, 所以软件测试员应和开发小组的程序员交流, 看看他们能否对其他的应该测试的次边界条件提供建议。

5.4.2.1 2的幂

计算机和软件的基础是二进制数——用位 (bit) 来表示0和1, 一个字节 (byte) 由8位组成, (在32位系统上) 一个字 (words) 由4个字节组成, 等等。表5-1列出了常用的2的幂单位及其等价数值。

表5-1 软件中2的幂

幂 单 位	范 围 或 值
Bit	0 or 1
Nibble	0-15
Byte	0-255
Word	0-4, 294, 967, 295
Kilo	1, 024
Mega	1, 048, 576
Giga	1, 073, 741, 824
Tera	1, 099, 511, 627, 776

表5-1所列的范围和值是作为边界条件的重要数据。除非软件向用户表示出同样的范围, 否则在需求文档中不会明确指明。然而, 它们通常由软件内部使用, 外部是看不见的, 当然除了产生软件缺陷的情况。

2的幂的示例

通信软件是可以体现2的幂的示例。带宽或者传输信息的能力总是受限制的, 人们总是需要尽可能更快地收发信息。因此, 软件工程师要尽一切努力在通信字符串中压缩更多的数据。

其中一个方法是把信息压缩到尽可能小的单元中, 发送这些小单元中最常用的信息, 在必要时扩展为大一些的单元。

假设某种通信协议支持256条命令。软件将发送编码为一个4位数据的最常用的15条命令。假如要用到第16到256条之间的命令, 软件就转而发送编码为更长的字节的命令。

软件用户只知道可以执行256条命令; 不知道软件根据4位/字节的边界执行了专门的计算和不同的操作。

在建立等价划分时, 要考虑等价划分中是否需要包含2的幂的边界条件。例如, 如果软件接受用户输入1~1000范围内的数字, 谁都知道在合法区间中包含1和1000, 也许还要有2和999。为了覆盖任何可能的2的幂的次边界, 还要包含临近4位边界的14、15和16, 以及临近字节边界的254、255和256。

5.4.2.2 ASCII表

另一个常见的次边界条件是ASCII字符表。表5-2是ASCII表的部分清单。

表5-2 部分ASCII值表

字 符	ASCII 值	字 符	ASCII 值
Null	0	B	66
Space	32	Y	89
/	47	Z	90
0	48	[	91
1	49	'	96
2	50	a	97
9	57	b	98
:	58	y	121
@	64	z	122
A	65	{	123

注意，表5-2不是良好的、连续的列表。0~9的ASCII值是48~57。斜杠字符 (/) 在数字0的前面，而冒号字符 (:) 在数字9的后面。大写字母A~Z对应的ASCII值是65~90。小写字母对应的ASCII值是97~122。这些情况都代表次边界条件。

如果测试进行文本输入或文本转换的软件，在定义数据划分包含哪些值时，参考一下ASCII表是相当明智的。例如，如果测试的文本框只接受用户输入字符A~Z和a~z，就应该在非法划分中包含ASCII表中这些字符前后的值——@、[、'和{。

ASCII和UNICODE

尽管ASCII仍然是软件表示字符数据非常流行的方式，但是它正被称为统一编码(Unicode)的新标准取代。Unicode于1991年由统一编码联合会开发，以解决ASCII码无法表示所有书面语言字符的问题。

ASCII只使用8位，能表示256种不同的字符。Unicode使用16位，可以表示65 535种字符。目前已经为39 000多种字符指定了数值，其中21 000多种用于表示中国象形文字。

5.4.3 默认、空白、空值、零值和无

另一种看起来很明显的软件缺陷来源是当软件要求输入时——比如在文本框中——不是没有输入正确的信息，而是根本没有输入任何内容，可能单单按了Enter键。这种情况在产品说明书中常常忽视，程序员也经常遗忘，但是在实际使用中却时有发生。

好的软件会处理这种情况。它通常将输入内容默认为边界内的最小合法值，或者在合法划分中间的某个合理值；或者返回错误提示信息。

Windows画图程序的Attributes（属性）对话框（见图5-6）通常在Width和Height文本框中放入默认值。如果用户有意无意地将默认值删除，使文本框成为空白，然后单击OK按钮，结果如何？

理想情况是软件取某个合法的宽度和高度作为默认值来处理。如果没有这样做，就应该返回错误提示信息作为执行结果（见图5-7）。虽然错误提示信息“位图必须大于一个像素”

的描述性不够好，但这是另外一个问题。

技巧 一定要考虑建立处理默认值、空白、空值、零值或者无输入等条件的等价划分。

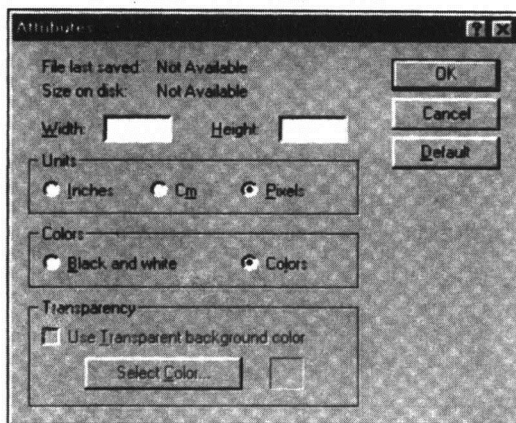


图5-6 Windows画图程序的Attributes（属性）对话框中Width和Height文本框空出

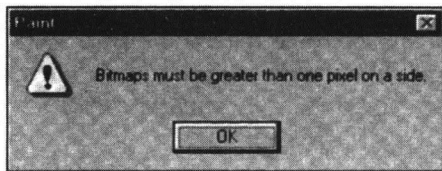


图5-7 在Width和Height文本框空出时按Enter键就会返回错误提示信息

因为这些值在软件中通常进行不同的处理，所以不要把它们与合法情况和非法情况混在一起，而要建立单独的等价划分。可能在这种默认情况下，和输入0或-1作为非法值比较起来，软件会执行不同的路径。由于考虑到软件的不同操作，所以应把这些作为单独的等价划分。

5.4.4 非法、错误、不正确和垃圾数据

数据测试的最后一种类型是垃圾数据。这是失效性测试的对象。经过边界测试、次边界测试和默认值测试等通过性测试证实软件能够工作之后，就该进行垃圾数据测试了。

从纯粹的软件测试观点来看，如果利用前述技术全面测试证明软件能够工作了，就不必再做破坏实验。然而，现实中考考虑到软件要应付用户千奇百怪的使用方式，这样做肯定没错。

如果想一想今天打包后的软件将售出数亿份拷贝，就完全可以断定一定有一部分用户会错误地使用软件。如果错误操作导致崩溃或者数据丢失，用户不会责怪自己——而会指责软件。软件如果没有按照用户的意愿运行，就算有一个缺陷，经常是这样。

非法、错误、不正确和垃圾数据测试是很有意思的。如果软件要求输入数字，就输入字母。如果软件只接受正数，就输入负数。如果软件对日期敏感，就看它在公元3 000年是否还能正常工作。假装有“肥胖的手指”，同时按下多个键。

· 此类测试没有实际的规则，只是设法破坏软件。要发挥创造力，要会走偏门。在此工作中寻找乐趣吧！

5.5 状态测试

到目前为止，我们测试的是数据——数字、文字、软件输入和输出。软件测试的另一方面是通过不同的状态验证的程序的逻辑流程。软件状态（software state）是指软件当前所处的条件或者模式，参见图5-8和图5-9。

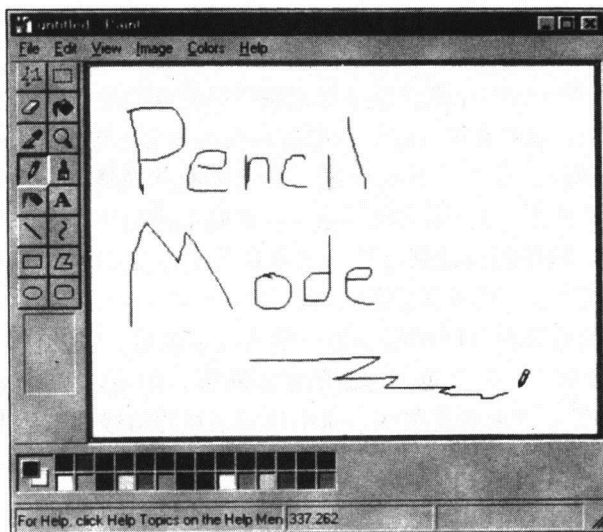


图5-8 处于铅笔绘画状态的Windows画图程序



图5-9 处于喷涂状态的Windows画图程序

图5-8显示了处于铅笔绘画状态的Windows画图程序，这是软件启动时的初始状态。注意，铅笔工具被选中，光标的形状很像铅笔，可以在屏幕上画出细线。图5-9显示了处于喷涂状态的Windows画图程序。在该状态下，喷枪工具被选中，喷枪大小确定，光标的形状很像喷漆罐，绘制效果很像喷漆。

进一步观察画图程序提供的全部选项——所有的工具、菜单、颜色等。一旦选中其中一项，使软件改变了外观、菜单或者某些操作，就是改变了该软件的状态。软件通过代码执行进入某一个分支，触发一些数据位，设置某些变量，读取某些数据，转入一个新的状态。

注意 软件测试员必须测试程序的状态及其转换。

5.5.1 测试软件的逻辑流程

还记得第3章中测试Windows计算器程序有无法穷尽的候选数据的例子吗？本章前面已经讲过，要使测试可以控制，就必须通过建立只包含最关键数字的等价划分来减少候选数据。

测试软件的状态和逻辑流程有同样的问题。访问所有状态通常是可以实现的（说到底，如果不能访问，还要它干什么？）。困难在于除了极其简单的程序之外，基本上不可能走遍所有分支，达到所有状态。软件的日益复杂化，尤其是为了迎合日益丰富的用户界面，提供了太多选择和选项，致使程序分支数量呈指数式增长。

这个问题与著名的流动推销员问题很相似：给定城市数目，以及任何两个城市之间的距离，设法找出访问每一个城市一次并返回起点的最短路线。如果只有5个城市，则可以快速计算出共有120条不同的路线。走遍所有路线，从中找到最短的路线并不是太难，花费不了太长时间。如果城市数目增加到成百上千——或者软件增加成百上千种状态——就形成一个难以解决的问题。

对于软件测试，解决方法是运用等价划分技术选择状态和分支。因为选择不做完全测试，所以要承担一定的风险，但是通过合理选择减少风险。

5.5.1.1 建立状态转换图

第一步是建立软件的状态转换图。这样的图可能作为产品说明的一部分被提供出来。如果是这样，则可以采用第4章“检查产品说明书”所述的技术进行静态测试。否则，就需要创建一个状态图。

绘制状态转换图有几种技术。图5-10给出了两个例子，一个使用方框和箭头。另一个使用圆圈（泡泡）和箭头。绘图使用的技术并不重要，只要项目小组中其他成员可以看懂就行了。

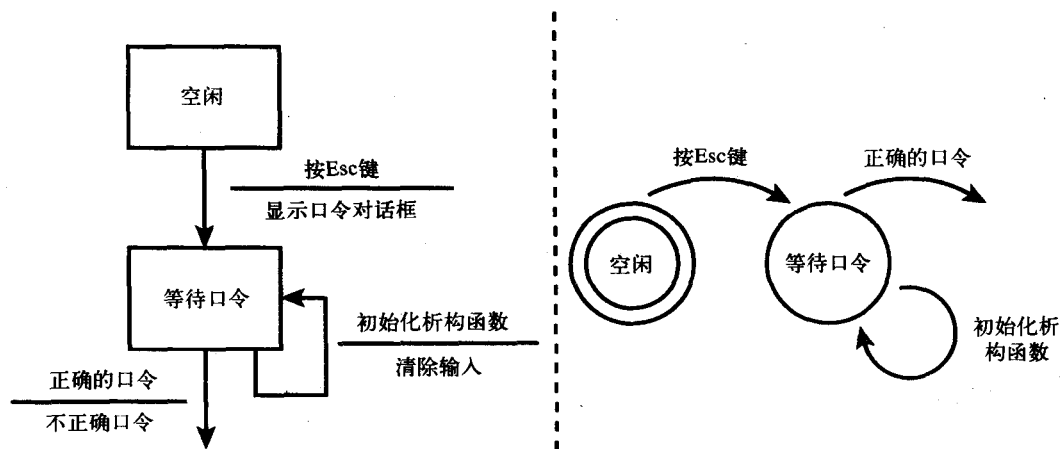


图5-10 状态转换图可以用不同的技术绘制

注意 状态转换图可能会变得非常庞大。许多开发小组在办公室墙上贴满了打印纸。

如果预计状态图会如此复杂，那么就找一些商业软件来绘制和管理。

状态转换图应该表示出以下项目：

- 软件可能进入的每一种独立状态。这里有一个很好的经验是，如果不能断定是否为独立状态，它就可能是。如果以后发现它不是，随时可以将其剔除。
- 从一种状态转入另一种状态所需的输入和条件。可能是按键、菜单选择、传感器信号或者电话振铃等。状态不可能无缘无故地存在，其原因正是我们在这里要寻找的。
- 进入或者退出某种状态时的设置条件及输出结果。包括显示的菜单和按钮、设置的标志位、产生的打印输出、执行的运算等。这些是状态转换时发生的部分或全部现象。

提示 因为正在进行黑盒测试，所以不必了解代码中设置的底层变量。从软件用户的角度建立状态图即可。

5.5.1.2 减少要测试的状态及转换的数量

为大型软件产品建立状态图是一项艰巨的任务。但愿只测试整个软件的一部分，使建立状态图变成一个可以接受的任务。完成状态图之后，就可以回过头来看看所有状态以及这些状态是如何转换的。圆满完成这项工作将是一个惊人之举。

如果有足够的时间，可能会想测试软件的每一个分支——不仅是连接两个状态的每一条线，而是每一种线路组合，翻来覆去、循环往复。与流动推销员问题一样，遍历所有的分支是不可能的。

正如对数据进行等价划分一样，需要将大量的可能性减少到可以操作的测试用例集合。有以下5种实现方法：

- 每种状态至少访问一次。如何到达的没有关系，但是每一种状态都必须测试。
- 测试看起来是最常见和最普遍的状态转换。尽管听起来很主观，但是其根据是进行产品说明书的静态黑盒子分析（见第3章）时收集到的信息。某些用户情况很可能比其他更常见。希望这样能管用。
- 测试状态之间最不常用的分支。这些分支是最容易被产品设计者和程序员忽视的。软件测试员也许是第一个测试它们的人。
- 测试所有错误状态及其返回值。出错条件通常难以建立。程序员常常编写代码处理某些错误，但不会测试自己的代码。错误没有得到正确处理、错误提示信息不正确、修复错误时未正确恢复软件等情况常有发生。
- 测试随机状态转换。如果打印了状态图，就可以在上面任意做各种标记。如果有时间做得更多，阅读第15章“自动测试和测试工具”关于如何自动执行状态随机转换测试。

5.5.1.3 怎样进行具体测试

确定要测试的状态及其转换之后，就可以定义测试用例了。

测试状态及其转换包括检查所有的状态变量（state variables）——与进入和退出状态相关的静态条件、信息、值、功能等。图5-11给出了Windows画图程序处于启动状态的例子。

以下是定义画图程序启动状态的部分状态变量：

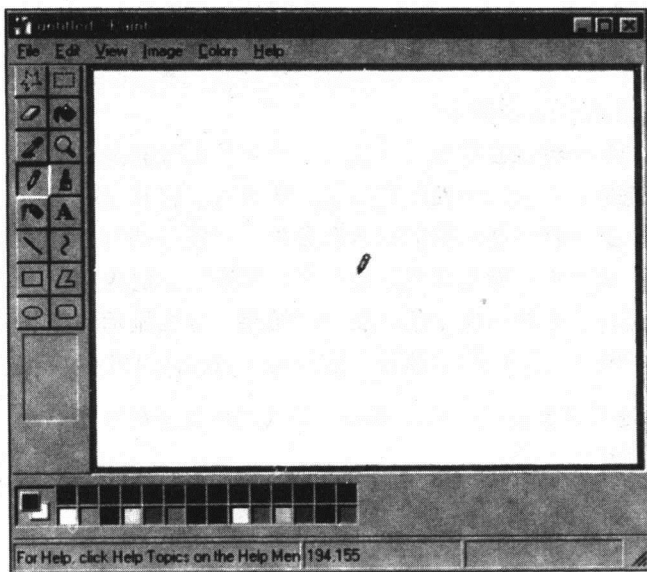


图5-11 Windows画图程序在启动状态打开的屏幕

- 如图5-11所示的窗口外观。
- 窗口尺寸被设置为上一次使用画图程序时的尺寸。
- 绘画区域空白。
- 显示工具栏、颜色栏和状态条。
- 铅笔工具被选中，而其他所有工具均未选中。
- 默认颜色是黑色前景、白色背景。
- 文档名称是untitled。

虽然仔细想想还能找出更多状态变量，但是以上这些变量已经能够为定义状态提供好的思路。请牢记，无论状态是看得见的窗口和对话框等，还是看不见的通信程序和金融软件包的组成部分等，都采用同样的过程来确定状态条件。

把对状态及其转换的假定与项目小组的产品说明书作者和程序员讨论是个好主意。他们可以提供软件测试员可能想不到的、表面现象背后的状态内幕。

文档涂改标志

状态变量也许看不见，但是很重要。一个常见的例子是文档涂改标志。

当文档载入编辑器，例如文字处理程序或画图程序中时，一个称为文档涂改标记（dirty document flag）的状态变量即被清除，软件处于“洁净”状态。只要文档未做任何修改，软件就保持这种状态。查看和滚动显示文档都不会改变这种状态。一旦用户键入任何内容，或者以某种方式修改文档，软件就转换到“涂改”状态。

在洁净状态下，试图关闭或者退出软件，软件会正常关闭。如果文档处于涂改状态，退出之前用户就会被询问是否保存所做的改动。

有些软件很高级，用户执行了涂改文档的编辑操作之后，可以撤销和恢复，使文档回到原来的情形，软件返回洁净状态。退出程序时不会提示保存文档。

5.5.2 失败状态测试

以上探讨的状态测试都属于通过性测试，测试包括审查软件、描绘状态、尝试各种合法可能性、确认状态及其转换正常。和数据测试一样，相反的做法是找到使测试软件失败的案例。此类案例的例子是竞争条件、重复、压迫和重负。

5.5.2.1 竞争条件和时序错乱

当今大多数操作系统，无论是用于个人计算机还是专用设备，都具备多任务执行能力。多任务（multitasking）是指操作系统设计用来同时执行多个独立的进程。这些进程可以是电子表格或者电子邮件这样的独立程序；也可以是同一个程序中的不同部分，例如在文字处理程序中，在后台打印的同时，允许用户输入。

设计多任务操作系统并不繁琐，设计充分利用多任务的软件才是艰巨的任务。在真正的多任务环境中，软件设计绝对不能想当然，必须处理随时被中断的情况，能够与其他任何软件在系统中同时运行，并且共享内存、磁盘、通信以及其他硬件资源。

这一切的结果就是可能导致竞争条件问题。这些问题是指几个事件恰巧挤在一起，由于软件未预料到运行过程会被中断，以致造成混乱。也就是说，时序发生错乱。竞争条件（race condition）一词源自很容易想到的情形——多个进程向终点线冲刺，不知道谁会首先到达。

注意 竞争条件测试难以设计，最好是首先仔细查看状态转换图中的每一个状态，以找出哪些外部影响会中断该状态。考虑要使用数据如果没有准备好，或者在用到时发生了变化，状态会怎样。数条弧线或者直线同时相连的情形如何？

以下是可能会面临竞争条件的例子情形：

- 两个不同的程序同时保存和打开同一个文档。
- 共享同一台打印机、通信端口或者其他外围设备。
- 当软件处于读取或者改变状态时按键或者单击鼠标。
- 同时关闭或者启动软件的多个实例。
- 同时使用不同的程序访问一个共同的数据库。

这看起来像胡乱测试，实际不是，用户常常意外地引起这些操作。软件必须足够强壮以应付此类情况。多年前这样的要求可能不合常理，但是现在，用户希望软件在此类情况下仍能正常工作。

5.5.2.2 重复、压迫和重负

另外三个失效性状态测试是重复、压迫和重负。这些测试的目标是那些处理程序员没考虑到，但在极端恶劣条件下可能发生问题的状态。

重复测试（repetition testing）是不断执行同样的操作。最简单的是不停地启动、关闭程序。还可以反复读写数据或者反复选择同一个操作。要想找出一个软件缺陷，可能只需重复几次操作，也可能需要成千上万次尝试。

进行这种反复测试的主要原因是检查是否存在内存泄漏（memory leaks）。如果计算机内存被分配进行某些操作，但是操作完成时没有完全释放，就会产生一个常见的软件问题。结果是最后程序耗尽了它赖以工作的内存空间。如果以前使用的某个程序在开始启动时工作状态良好，但是随后变得越来越慢，或者经过一段时间就表现不稳定，原因就可能是内存泄漏

缺陷。重复测试就能够暴露这些问题。

压迫测试 (stress testing) 是使软件在不够理想的条件下运行——内存小、磁盘空间少、CPU速度慢、调制解调器速率低等。观察软件对外部资源的要求和依赖的程度。压迫测试就是将支持降到最低限度, 目的在于尽可能地限制软件的必要条件。这是否有点像边界条件测试呢? 没错。

重负测试 (load testing) 与压迫测试相反。压迫测试是尽量限制软件, 而重负测试是尽量提供条件任其发挥。让软件处理尽可能大的数据文件。如果软件对打印机或者通信端口之类的外设进行操作, 就把能连的都连上。如果正在测试的因特网服务器可以处理几千个并发连接, 就按它说的做。最大限度地发掘软件的能力, 让它不堪重负。

不要忘了时间也是一种重负测试。对于大多数软件, 长期稳定地工作是很重要的。某些软件应该能够永远运行下去, 而不用重新启动。

注意 重复、压迫和重负测试应联合使用, 同时进行, 这是找出以其他方式难以发现的严重缺陷的一个可靠的方法。

对于重复、压迫和重负测试有两个重要事项:

- 项目经理和小组程序员可能不完全接受软件测试员这样破坏软件的做法。他们可能争辩客户不会这样使用系统, 或者强调这是软件测试员的看法。事实上不然, 客户会这样做的。软件测试员的任务是确保软件在这样恶劣的条件下仍能正常工作, 否则就报告软件缺陷。第19章“报告发现的问题”将讲述如何以最佳方式报告软件缺陷, 使其得到认真对待和修复。
- 无数次打开和关闭程序对于手工操作是不可能的。同样, 找出几千人与因特网服务器连接也是难以实现的。第15章讲述了测试自动化, 以及如何进行此类测试而不需兴师动众的技巧。

5.6 其他黑盒测试技术

余下的黑盒测试技术不像已经描述的数据测试和状态测试那样独立, 而是它们的变形。如果对整个程序数据进行了完整的等价划分, 创建了详细的状态图, 并且开发完成了相关的测试用例, 就会发现大多数能由用户发现的软件缺陷。

剩下的是找出缺陷中的漏网之鱼的技术。假如这些软件缺陷是有生命的, 就应该有自己的思想和行为方式。要找出它们也许有一点主观, 没有实实在在的理由根据, 但是如果要找出所有的软件缺陷, 就必须得有一点创造力。

5.6.1 像笨拙的用户那样做

为了礼貌一些, 正确的说法也许应该是无经验的用户 (inexperienced user) 或新用户 (new user), 但是事实上都是一回事。一个不熟悉软件的人面对程序时, 他会做出令人永远想不到的举动。他们会输入程序员无从想像的数据。他们会在中途变卦, 退回去执行其他操作。他们冲浪遇到某个站点, 可能会单击不应该单击的东西。他们会发现开发小组完全遗漏的软件缺陷。

软件测试员看到一个没有任何测试经验的人只花5分钟来使用软件并使其崩溃, 一定会感

到沮丧吧！他们是怎样做的？他们不遵循任何规则，也不做任何假定。

在设计测试用例或者初次查看软件时，要设法像笨拙的用户那样想问题。抛开关于软件应该如何工作的先入之见。如果可能，找一个其他专业的朋友来整理思路。假设他什么也不会。把这些测试用例加入到已经设计好的测试用例库中，就会更加全面。

5.6.2 在已经找到的软件缺陷的地方再找找

在已经找到软件缺陷的地方再找的原因有两个：

- 如第3章所述，找到的软件缺陷越多，就说明那里的软件缺陷越多。如果发现在不同的特性中找出了大量上边界条件软件缺陷，那么明智的做法是对所有特性着重测试上边界条件。当然无论如何要进行此类测试的，但是还应该投入一些案例来保证这个问题不是普遍存在的。
- 许多程序员倾向于只修复报告出来的软件缺陷，不多也不少。如果报告软件缺陷是启动—终止—再启动255次导致崩溃，程序员就只修复这个问题。也许是内存泄漏导致这个问题出现，程序员找到症结并将其修复。当拿回软件重新测试时，一定要重新执行同样的测试256次以上。在这个范围之外极有可能存在其他的内存泄露问题。

5.6.3 像黑客一样考虑问题

在第13章将了解到，没有软件是100%安全的。黑客知道这一点，会寻找软件的漏洞并利用这些漏洞。作为测试员，需要从另外的角度考虑问题。想想软件里面有哪些有价值的东西，为什么有人要想获得其访问权限，黑客进入的方法有哪些。不要太绅士，黑客不会绅士。

5.6.4 凭借经验、直觉和预感

要想成为真正的软件测试员，积累经验是不可替代的。没有比亲自动手更好的学习工具，也没有比客户第一次打电话报告刚经过测试的软件中存在缺陷一事更好的教训了。

经验和直觉是不可言传的，必须经过长期的积累。运用现在学到的全部技术进行测试，仍然有可能遗漏重要的软件缺陷。这是无法更改的事实。随着在职业生涯中逐步提高，学习测试不同类型和规模的产品，就会得到各种提示和技巧以便更加有效地找出令人棘手的软件缺陷。重新开始测试新软件，就可以很快找出以前同事可能遗漏的软件缺陷。

记录哪些技术有效，哪些不行。尝试不同的途径。如果认为有可疑之处，要深入探究。按照预感行事，直至证实这是错误为止。

经验是每个人为其错误寻找的代名词。

——奥斯卡·王尔德 (Oscar Wilde)

5.7 小结

本章够长的。动态黑盒测试范围广泛。对于测试新手而言，这也是最重要的一章。这好比是面试或者上岗第一天接到软件进行测试。运用本章所讲的技术是快速找出软件缺陷行之有效的方法。

然而，不能以为这就是软件测试的全部内容。如果是这样，本书其他各章就不用看了。动态黑盒测试只是引入大门。软件测试还有很多内容，现在只是刚开始。

以下两章介绍访问程序代码时进行的软件测试，讲述代码的工作方式和底层特性。上述黑盒子技术仍然有效，但是要用新技术来完善，从而成为更称职的软件测试员。

5.8 小测验

以下是帮助读者加深理解的小测验。答案参见附录A——但是不要偷看！

1. **判断是非**：在没有产品说明书和需求文档的条件下可以进行动态黑盒测试。
2. 如果测试程序向打印机输送打印内容，应该选用哪些通用的失效性测试用例？
3. 启动Windows写字板程序，并从File菜单选取Print命令，打开如图5-12所示的对话框。左下角显示的Print Range（打印区域）特性存在什么样的边界条件？

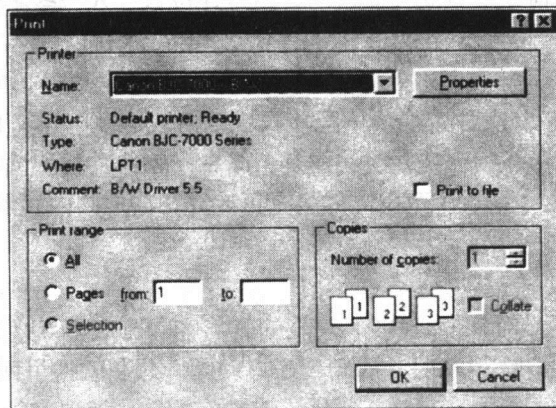


图5-12 Windows打印对话框显示了打印区域特性

4. 假设有一个文本框要求输入10个字符的邮政编码，如图5-13所示。对于该文本框应该进行怎样的等价划分？

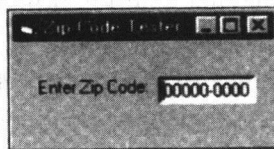


图5-13 最多允许输入10个字符的示范邮政编码文本框

5. **判断是非**：访问程序的所有状态也确保了遍历各种状态之间的转换。
6. 绘制状态转换图有多种不同的方法，但是它们都具有相同的三个要素是什么？
7. Windows计数器程序的初始状态变量有哪些？
8. 当设法显露竞争条件软件缺陷时，要对软件进行何种操作？
9. **判断是非**：在进行压迫测试的同时进行重负测试是不合情理的。